

Практическая работа №2

Инженерное измерение RTT и анализ качества UDP-соединения

Практическая работа продолжает ПР №1, где созданы UDP-клиент/сервер, заголовок пакета и команды `MOVEMENT` / `SHOOT`. Здесь создаётся изолированная подсистема телеметрии сети. Она будет использована в ПР №3 при реализации ACK, тайм-аутов и повторных передач.

Цель

Спроектировать и реализовать подсистему телеметрии UDP-протокола: измерять RTT, оценивать сглаженный RTT, джиттер и долю потерь, корректно обрабатывать недействительные, дублирующиеся и запоздалые ответы. Оцениваются качество модульной архитектуры, тестируемость, проверка входных данных и воспроизводимость эксперимента.

Результат

В репозитории должны быть:

- UDP-клиент и UDP-сервер с пакетами `PING` / `PONG` ;
- модули `protocol` , `telemetry` , `transport` , не смешивающие сетевой ввод, сериализацию и расчёт метрик;
- журнал измерений `docs/latency_samples.csv` ;
- автоматические тесты сериализации, статистики и ошибочных пакетов;
- `docs/Experiment_Config.md` и `docs/Latency_Report.md` с графиками;
- ветка `feature/latency-measurement` , issues, Pull Request и ревью участника команды.

Задание

Контракт протокола

Добавьте типы `PING` (клиент → сервер) и `PONG` (сервер → клиент). В каждом пакете должны быть версия протокола, порядковый номер и размер полезной нагрузки.

Поле	Тип	Назначение
<code>packetType</code>	<code>uint8_t</code>	Тип пакета: <code>PING</code> , <code>PONG</code> , <code>MOVEMENT</code> , <code>SHOOT</code>
<code>sequenceNumber</code>	<code>uint16_t</code>	Идентификатор измерения
<code>payloadSize</code>	<code>uint16_t</code>	Размер полезных данных в байтах
<code>protocolVersion</code>	<code>uint16_t</code>	Версия формата пакета

`PING` содержит `clientSendTimeUs` (`uint64_t`). `PONG` содержит исходную метку клиента, а также `serverReceiveTimeUs` и `serverSendTimeUs` (`uint64_t`). Часы сервера и клиента не синхронизируются: серверные метки применяются только для диагностики, а RTT вычисляется исключительно по времени клиента.

Обязательно: не сериализуйте `struct` побайтовым копированием. Реализуйте явные `WriteU16` , `WriteU64` , `ReadU16` , `ReadU64` в сетевом порядке байтов. До чтения проверяйте длину заголовка, `packetType` , версию, совпадение `payloadSize` с длиной датаграммы и границы буфера. Некорректная датаграмма отбрасывается без падения процесса и отражается в логе.

Телеметрия клиента

- Клиент создаёт `PING` с новым `sequenceNumber` , сохраняет запись в ограниченном контейнере `inFlight` .
- Запись содержит номер, время отправки, идентификатор серии, статус и число попыток.
- При `PONG` с известным номером вычисляется:

[

$$RTT = t_{\text{receive}}^{\text{client}} - t_{\text{send}}^{\text{client}}$$

]

- Сглаженная оценка обновляется как `SRTT = 0.875 * SRTT + 0.125 * RTT` ; первое успешное RTT инициализирует `SRTT` .

2. Тайм-аут — 1 000 мс либо обоснованное в отчёте значение. Поздний ответ получает статус `late_response` ; повторный — `duplicate_response` ; неизвестный номер — `unknown_response` .
3. Повторная отправка в ПР №2 не выполняется: она относится к следующей работе.
- Используйте монотонные часы: `std::chrono::steady_clock` в C++ или `Stopwatch` в C#.

Логи и метрики

Минимальный CSV:

```
1 experiment_id;sample;sequence;sent_at_ms;rtt_ms;srtt_ms;status
2 baseline;1;42;0.000;1.73;1.73;received
3 loss_5;8;57;7000.000;;;timeout
```

Для каждой серии рассчитайте число отправленных/полученных/просроченных пакетов, min/max/mean/median RTT, `SRTT` , джиттер и потери. Джиттер:

$$J = \frac{1}{n - 1} \sum_{i=2}^n |RTT_i - RTT_{i-1}|$$

Потери:

$$\text{Loss;Rate} = \frac{N_{\text{timeout}}}{N_{\text{sent}}} \times 100\%$$

Эксперимент

Каждая серия содержит минимум 50 `PING` с постоянным интервалом 200–1000 мс. Используйте Clumsy или согласованный эмулятор; если задержка встроена в сервер, это явно фиксируется в отчёте.

Серия	Условия
<code>baseline</code>	Без искусственных ограничений
<code>delay_50</code>	Дополнительная задержка 50 мс
<code>delay_100</code>	Дополнительная задержка 100 мс

<code>jitter</code>	Флуктуации, например 50–150 мс
<code>loss_5</code>	Потери 5%
<code>combined</code>	100 мс задержки, джиттер и 5% потерь

В `Experiment_Config.md` зафиксируйте ОС, адрес/порт, инструмент и его версию, параметры, интервал, дату, число пакетов и seed генератора, если он доступен.

Постройте: линейный график RTT по измерениям с тайм-аутами, столбчатую диаграмму mean RTT/SRTT/loss rate, а также histogram или box plot для `baseline`, `jitter`, `combined`.

Примеры реализации (C++17)

Примеры иллюстрируют подход; студент обязан интегрировать их в собственную архитектуру и объяснить код на защите.

Явная сериализация PING

```

1 #include <cstdint>
2 #include <optional>
3 #include <vector>
4
5 constexpr std::uint16_t kProtocolVersion = 1;
6 enum class PacketType : std::uint8_t { Ping = 1, Pong = 2, Movement = 3, Shoot
  = 4 };
7
8 void WriteU16(std::vector<std::uint8_t>& out, std::uint16_t v) {
9     out.push_back(static_cast<std::uint8_t>(v >> 8));
10    out.push_back(static_cast<std::uint8_t>(v));
11 }
12 void WriteU64(std::vector<std::uint8_t>& out, std::uint64_t v) {
13     for (int s = 56; s >= 0; s -= 8) out.push_back(static_cast<std::uint8_t>(v
14 >> s));
15 }
16 std::optional<std::uint16_t> ReadU16(const std::uint8_t* p, const
17 std::uint8_t* end) {
18     if (end - p < 2) return std::nullopt;
19     auto v = static_cast<std::uint16_t>((p[0] << 8) | p[1]); p += 2; return v;
20 }
21 std::optional<std::uint64_t> ReadU64(const std::uint8_t* p, const
22 std::uint8_t* end) {

```

```

20     if (end - p < 8) return std::nullopt;
21     std::uint64_t v = 0; for (int i = 0; i < 8; ++i) v = (v << 8) | *p++;
    return v;
22 }
23 std::vector<std::uint8_t> SerializePing(std::uint16_t seq, std::uint64_t
    sentUs) {
24     std::vector<std::uint8_t> out; out.reserve(15);
25     out.push_back(static_cast<std::uint8_t>(PacketType::Ping));
26     WriteU16(out, seq); WriteU16(out, 8); WriteU16(out, kProtocolVersion);
    WriteU64(out, sentUs);
27     return out;
28 }
29 std::optional<std::uint64_t> ParsePing(const std::uint8_t* data, std::size_t
    size) {
30     if (size != 15) return std::nullopt;
31     const auto* p = data; const auto* end = data + size;
32     if (static_cast<PacketType>(*p++) != PacketType::Ping) return
    std::nullopt;
33     auto seq = ReadU16(p, end); auto length = ReadU16(p, end); auto version =
    ReadU16(p, end);
34     if (!seq || !length || !version || *length != 8 || *version !=
    kProtocolVersion) return std::nullopt;
35     auto sent = ReadU64(p, end); if (!sent || p != end) return std::nullopt;
36     return sent;
37 }

```

Учёт RTT, SRTT и состояний ответа

```

1 #include <cmath>
2 #include <stdint>
3 #include <unordered_map>
4
5 enum class ResponseStatus { Received, Timeout, Late, Duplicate, Unknown };
6 struct Flight { std::uint64_t sentUs; bool completed = false; };
7
8 class Telemetry {
9 public:
10     void OnPingSent(std::uint16_t seq, std::uint64_t nowUs) { inFlight_[seq] =
    {nowUs}; }
11     ResponseStatus OnPong(std::uint16_t seq, std::uint64_t nowUs) {
12         auto it = inFlight_.find(seq);
13         if (it == inFlight_.end()) return ResponseStatus::Unknown;
14         if (it->second.completed) return ResponseStatus::Duplicate;
15         const double rtt = (nowUs - it->second.sentUs) / 1000.0;
16         if (nowUs - it->second.sentUs > kTimeoutUs) { it->second.completed =
    true; return ResponseStatus::Late; }

```

```

17     if (hasPrevious_) jitterSum_ += std::abs(rtt - previousRtt_);
18     previousRtt_ = rtt; hasPrevious_ = true;
19     srtt_ = samples_ == 0 ? rtt : 0.875 * srtt_ + 0.125 * rtt;
20     ++samples_; it->second.completed = true; return
    ResponseStatus::Received;
21 }
22 std::size_t Expire(std::uint64_t nowUs) {
23     std::size_t result = 0;
24     for (auto& [seq, f] : inFlight_) if (!f.completed && nowUs - f.sentUs
    > kTimeoutUs) { f.completed = true; ++result; }
25     return result;
26 }
27 double SrttMs() const { return srtt_; }
28 double MeanJitterMs() const { return samples_ > 1 ? jitterSum_ / (samples_
    - 1) : 0.0; }
29 private:
30     static constexpr std::uint64_t kTimeoutUs = 1'000'000;
31     std::unordered_map<std::uint16_t, Flight> inFlight_;
32     double srtt_ = 0, previousRtt_ = 0, jitterSum_ = 0; std::size_t samples_ =
    0; bool hasPrevious_ = false;
33 };

```

Пример тестов Catch2

```

1 #include <catch2/catch_test_macros.hpp>
2
3 TEST_CASE("PING round trip") {
4     auto bytes = SerializePing(17, 987654321ULL);
5     auto sent = ParsePing(bytes.data(), bytes.size());
6     REQUIRE(sent.has_value());
7     REQUIRE(*sent == 987654321ULL);
8 }
9 TEST_CASE("Truncated PING is rejected") {
10     auto bytes = SerializePing(17, 100);
11     REQUIRE_FALSE(ParsePing(bytes.data(), bytes.size() - 1).has_value());
12 }

```

План выполнения

Семинар	Время	Этап	Действия
---------	-------	------	----------

3	15 мин	Проектирование	Контракт пакетов, диаграмма состояний <code>inFlight</code> , issues, ветка <code>feature/latency-measurement</code> .
3	45 мин	Реализация	Модули protocol/telemetry/transport, PING/PONG, валидация, RTT/SRTT, логирование.
3	30 мин	Базовый тест	50 baseline-измерений, CSV, тесты сериализации и статистики.
4	20 мин	Конфигурация	Документирование параметров эмулятора и критериев тайм-аута.
4	40 мин	Эксперимент	Пять оставшихся серий, контроль логов сервера и ошибочных ответов.
4	30 мин	Анализ	Метрики, три визуализации, отчёт, PR, ревью и

Структура репозитория

▼

1

client/ server/ protocol/ telemetry/ tests/

2

docs/Protocol_Specification.md

3

docs/Experiment_Config.md

4

docs/latency_samples.csv

5

docs/graphs/

6

docs/Latency_Report.md